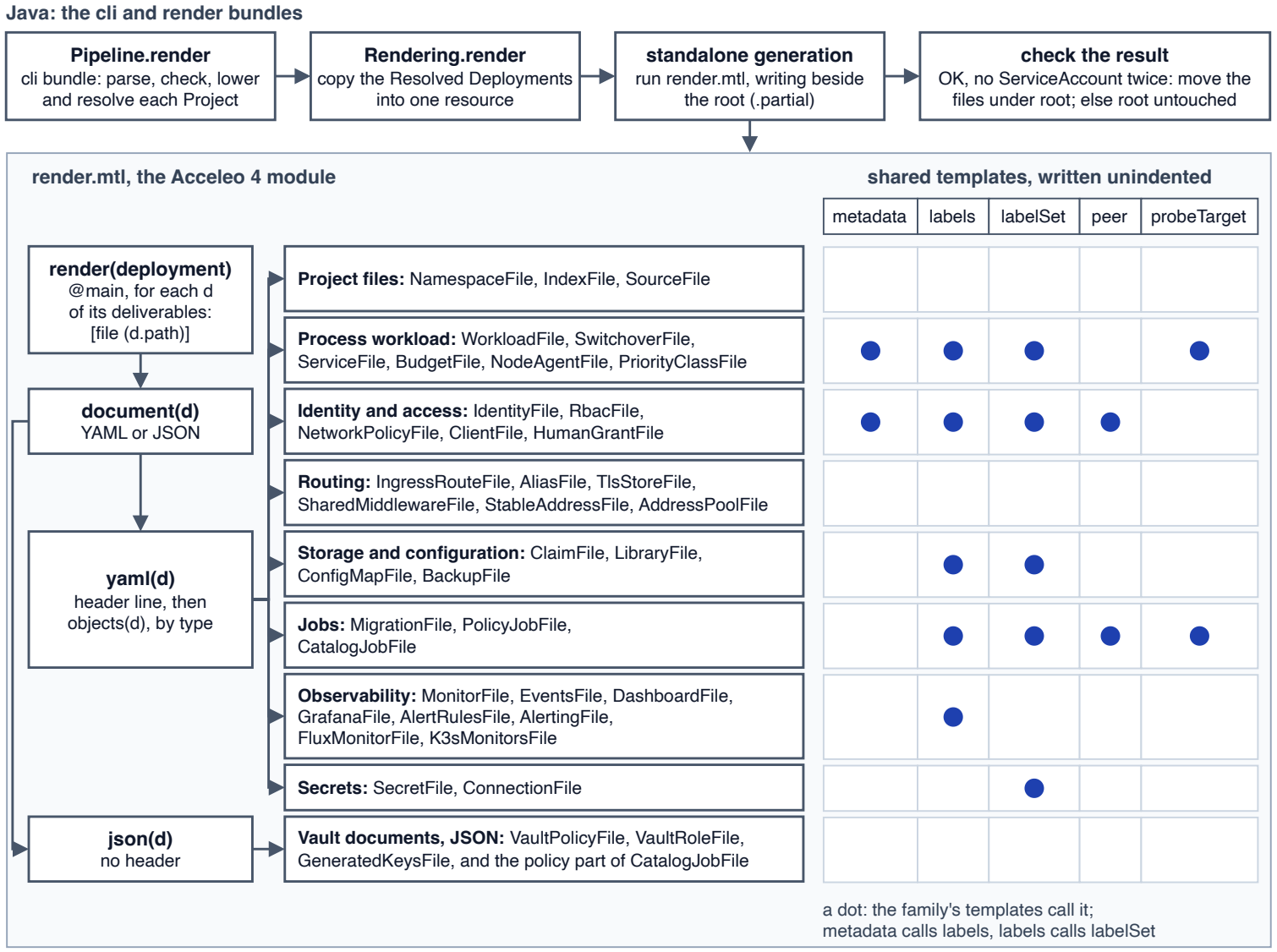
Java: the cli and render bundles
Pipeline.render
cli bundle: parse, check, lower and resolve each Project
Rendering.render
copy the Resolved Deployments into one resource
standalone generation
run render.mtl, writing beside the root (.partial)
check the result
OK, no ServiceAccount twice: move the files under root; else root untouched
render.mtl, the Acceleo 4 module
shared templates, written unindented
metadata
labels
labelSet
peer
probeTarget
render(deployment)
@main, for each d of its deliverables: [file (d.path)]
document(d)
YAML or JSON
yaml(d)
header line, then objects(d), by type
json(d)
no header
Project files: NamespaceFile, IndexFile, SourceFile
Process workload: WorkloadFile, SwitchoverFile, ServiceFile, BudgetFile, NodeAgentFile, PriorityClassFile
Identity and access: IdentityFile, RbacFile, NetworkPolicyFile, ClientFile, HumanGrantFile
Routing: IngressRouteFile, AliasFile, TlsStoreFile, SharedMiddlewareFile, StableAddressFile, AddressPoolFile
Storage and configuration: ClaimFile, LibraryFile, ConfigMapFile, BackupFile
Jobs: MigrationFile, PolicyJobFile, CatalogJobFile
Observability: MonitorFile, EventsFile, DashboardFile, GrafanaFile, AlertRulesFile, AlertingFile, FluxMonitorFile, K3sMonitorsFile
Secrets: SecretFile, ConnectionFile
Vault documents, JSON: VaultPolicyFile, VaultRoleFile, GeneratedKeysFile, and the policy part of CatalogJobFile
a dot: the family's templates call it;
metadata calls labels, labels calls labelSet